KPP Python Port Reference: Code Flow and Fortran-to-Python Mapping

1D Mixing Model Development Team

August 2026

Contents

1 Introduction

1.1 Purpose of This Document

This document serves as a **reference guide** for mapping the MITgcm KPP (K-Profile Parameterization) mixing scheme (Fortran) to its Python port in the 1D Column Model. The document is organized by algorithm flow, providing file and line number references for each major computational step. Use this document to:

- Trace where specific KPP capabilities from MITgcm are implemented in Python
- Understand the computational flow of the KPP mixing scheme
- Locate corresponding code sections when comparing Fortran and Python implementations
- Identify file-to-file correspondence between the two implementations

1.2 Scope of the Port

The Python port implements the KPP boundary layer parameterization for a 1D ocean column. The port faithfully reproduces the core physics and numerical methods from MITgcm's `pkg/kpp`, adapted for the 1D vertical column context.

Included in the port:

- Boundary layer depth diagnosis (bulk Richardson criterion)
- Surface forcing calculation (friction velocity, buoyancy forcing)
- Interior Richardson number-based mixing
- Boundary layer mixing profiles (cubic shape functions)
- Nonlocal transport (counter-gradient term)
- Diffusivity assembly (interior + boundary layer)
- Enhancement at boundary layer interface
- Stratification and shear diagnostics
- Shortwave penetration (single Jerlov water type)

Excluded from the 1D port:

- Horizontal mixing and advection
- Horizontal smoothing of buoyancy gradients (requires 2D/3D data)
- Double diffusion parameterization (not critical for 1D validation)
- Salt plume parameterization
- Multiple Jerlov water types (simplified to single type)
- 3D-specific features (requires horizontal neighbors)

1.3 How to Use This Document

1. **Section 2 (Overview):** Start here to understand the high-level algorithm flow and main entry points
2. **Section 3 (Step-by-Step Mapping):** Use this to find where each major computational step lives in both codebases
3. **Section 4 (Infrastructure):** Consult for parameter handling, grid conventions, and boundary conditions
4. **Section 5 (File Correspondence):** Quick lookup table for file-to-file mapping
5. **Section 6 (Variable Reference):** Look up variable name mappings and sign conventions

2 Overview of Code Flow

2.1 Algorithm Flowchart

The KPP scheme computes vertical mixing coefficients through the following sequence:

1. **Parameter Initialization**: Load scheme parameters from configuration files
2. **Density and Buoyancy Calculation**: Compute density, buoyancy gradients, and thermal expansion coefficients
3. **Boundary Layer Depth Diagnosis**: Find boundary layer depth using bulk Richardson criterion
4. **Surface Forcing Calculation**: Compute surface friction velocity and buoyancy forcing
5. **Interior Richardson Mixing**: Compute diffusivity below boundary layer based on local Ri
6. **Boundary Layer Mixing**: Compute mixing within boundary layer using shape functions
7. **Nonlocal Transport**: Calculate counter-gradient flux term
8. **Diffusivity Assembly**: Combine interior and boundary layer contributions
9. **Enhancement at Interface**: Enhance mixing at boundary layer base

2.2 Main Entry Points

Python:

- Module: `KPP_ML/KPP_PY/kpp_core_driver.py`
- Class: `KPPDriver` (lines 80-400+)
- Method: `compute_mixing()` (lines 105-400+)
- Called from: `main/mixing_adapter.py` via unified interface

Fortran:

- File: `pkg/kpp/kpp_calc.F`
- Subroutine: `KPP_CALC` (lines 19-799)
- File: `pkg/kpp/kpp_routines.F`
- Subroutine: `KPPMIX` (lines 28-1765)
- Called from: `model/src/do_oceanic_phys.F`

2.3 Calling Sequence

Python calling sequence:

```
unified_driver.py::step_forward()
  -> mixing_adapter.py::compute_mixing_coefficients()
    -> kpp_core_driver.py::KPPDriver.compute_mixing()
      -> eos.py::compute_buoyancy_gradients()
      -> kpp_core_driver.py::_compute_surface_forcing()
      -> kpp_core_driver.py::_compute_shear()
      -> kpp_routines.py::ri_iwmix()
      -> kpp_scheme_specific.py::diagnose_bl_depth()
      -> kpp_scheme_specific.py::compute_bl_mixing()
      -> kpp_scheme_specific.py::enhance_at_interface()
```

Fortran calling sequence:

```
do_oceanic_phys.F::DO_OCEANIC_PHYS()
  -> kpp_calc.F::KPP_CALC()
    -> kpp_forcing_surf.F::KPP_FORCING_SURF()
    -> kpp_routines.F::STATEKPP()
    -> kpp_routines.F::KPPMIX()
      -> kpp_routines.F (internal)::BLDEPTH()
      -> kpp_routines.F (internal)::WSCALE()
      -> kpp_routines.F (internal)::RI_IWMIX()
      -> kpp_routines.F (internal)::BLMIX()
      -> kpp_routines.F (internal)::ENHANCE()
```

3 Step-by-Step Code Mapping

This section provides detailed line-by-line mapping for each major computational step in the KPP algorithm.

3.1 Parameter Initialization

Purpose: Load and initialize KPP scheme parameters, including critical Richardson number (Ri_c), shape function coefficients, velocity scale parameters, and numerical control flags.

Python Implementation:

- File: `KPP_ML/KPP_PY/kpp_parameters.py`

- Class: `KPPParameters` (lines 1-300+)
- Configuration files: `configuration_yamls/kpp_default.yaml`
- Key parameters loaded: `Ricr`, `cstar`, `epsilon`, `Vtc`, `nu0`, `diff_min`, `diff_max`

Fortran Source:

- File: `pkg/kpp/kpp_readparms.F` (lines 1-250+)
- Subroutine: `KPP_READPARMS` reads namelist `KPP_PARM01`
- File: `pkg/kpp/kpp_init_fixed.F` (lines 1-200+)
- Subroutine: `KPP_INIT_FIXED` initializes fixed parameters
- Header: `pkg/kpp/KPP_PARAMS.h` defines parameter variables

Key Parameters:

- `KPPRicr` (Fortran) ↔ `Ricr` (Python): Critical bulk Richardson number (default: 0.3)
- `KPPcstar` (Fortran) ↔ `cstar` (Python): Shape function coefficient (default: 10.0)
- `KPPepsilon` (Fortran) ↔ `epsilon` (Python): Nondim. extent of surface layer (default: 0.1)
- `KPPVtc` (Fortran) ↔ `Vtc` (Python): Unresolved turbulence parameter (default: 1.8)
- `KPPnu0` (Fortran) ↔ `nu0` (Python): Maximum diffusivity (default: 5.0e-3 m^2/s)

3.2 Density and Buoyancy Calculation

Purpose: Compute density, buoyancy gradients (dbloc), surface buoyancy relative to depth (Ritop), and thermal expansion/haline contraction coefficients. These are fundamental inputs for Richardson number and boundary layer depth diagnosis.

Python Implementation:

- File: `main/eos.py`
- Function: `compute_buoyancy_gradients()` (lines 1-200+)
- Called from: `kpp_core_driver.py::compute_mixing()` (lines 184-186)
- Returns: `rho_surf`, `dbloc`, `dbsfc`, `ttalpha`, `ssbeta`
- Uses JMD95 equation of state (simplified for 1D)

Fortran Source:

- File: `pkg/kpp/kpp_routines.F`
- Subroutine: `STATEKPP` (lines 1766-1961)
- Called from: `kpp_calc.F::KPP_CALC` (line 300+)
- Computes: `rho1`, `dbloc`, `dbsfc`, `ttalpha`, `ssbeta`

- Uses model equation of state (`find_rho.F`)

Key Computations:
Local buoyancy gradient across interface k:

$$\text{dbloc}[k] = g\frac{\rho[k] - \rho[k-1]}{\rho_0} \tag{1}$$

Surface buoyancy difference (numerator of bulk Ri):

$$\text{Ritop}[k] = (z_{\text{ref}} - z[k]) \times g\frac{\rho[k] - \rho_{\text{surf}}}{\rho_0} \tag{2}$$

Implementation Notes:

- Python uses JMD95 simplified equation of state
- Fortran calls full MITgcm EOS (can be linear, JMD95, MDJWF, etc.)
- Both compute potential density (not in-situ) for consistency

3.3 Boundary Layer Depth Diagnosis

Purpose: Diagnose the ocean boundary layer depth (hbl) by finding the depth where the bulk Richardson number exceeds a critical threshold. This determines the extent of surface-driven turbulent mixing.

Python Implementation:

- File: `KPP_ML/KPP_PY/kpp_scheme_specific.py`
- Function: `diagnose_bl_depth()` (lines 26-200+)
- Called from: `kpp_core_driver.py::compute_mixing()` (line 210+)
- Key computation: lines 94-144 (Rib loop and comparison to Ricr)
- Interpolation: lines 146-180+ (linear interpolation to find exact hbl)
- Returns: `hbl`, `bfsfc`, `stable`, `casea`, `kbl`, `Rib`

Fortran Source:

- File: `pkg/kpp/kpp_routines.F`
- Function: `BLDEPTH` (internal to KPPMIX, lines 600-800+)
- Called from: `KPPMIX` (line 400+)
- Richardson number loop: lines 630-700+
- Interpolation: lines 660-680+
- Returns: `hbl(i)`, `kbl(i)`

Key Computation:
Bulk Richardson number at depth z:

$$\mathrm{Ri}_b(z) = \frac{\mathrm{Ritop}(z)}{\mathrm{dvsq}(z) + V_t^2 + \phi_\epsilon} \tag{3}$$

where:

- $\mathrm{Ritop}(z) = (z_{\mathrm{ref}} - z) \times$ buoyancy difference from surface
- $\mathrm{dvsq}(z)$ = velocity shear squared relative to surface
- V_t^2 = unresolved turbulent shear contribution
- ϕ_ϵ = small regularization parameter

Boundary layer depth is where $\mathrm{Ri}_b = \mathrm{Ri}_c$ (typically 0.3):

$$\mathrm{hbl} = z \quad \text{where} \quad \mathrm{Ri}_b(z) = \mathrm{Ri}_c \tag{4}$$

Implementation Notes:

- Linear interpolation between grid levels to find exact hbl
- Shortwave penetration reduces effective surface buoyancy forcing at depth
- Python port includes July 2026 fix for off-by-one interpolation guard
- Stable vs. unstable forcing determines velocity scale computation

3.4 Surface Forcing Calculation

Purpose: Compute surface friction velocity (u_*) and surface buoyancy forcing (bo, bosol) from atmospheric fluxes. These drive the boundary layer turbulence.

Python Implementation:

- File: `KPP_ML/KPP_PY/kpp_core_driver.py`
- Method: `KPPDriver._compute_surface_forcing()` (lines 250-320+)
- Called from: `compute_mixing()` (lines 193-196)
- Friction velocity: $u_*^2 = \sqrt{\tau_x^2 + \tau_y^2}$ (line 260+)
- Buoyancy forcing: `bo` (turbulent) and `bosol` (radiative) (lines 280-310+)

Fortran Source:

- File: `pkg/kpp/kpp_forcing_surf.F`
- Subroutine: `KPP_FORCING_SURF` (lines 1-500+)
- Called from: `kpp_calc.F::KPP_CALC` (line 150+)
- Friction velocity: lines 200-250+ (from wind stress)

- Buoyancy forcing: lines 300-400+ (from heat/freshwater fluxes)

Key Computations:
Friction velocity:

$$u_* = \sqrt[4]{\tau_x^2 + \tau_y^2} \tag{5}$$

Turbulent buoyancy forcing:

$$\text{bo} = g\left(\alpha \frac{Q_{\text{net}} - Q_{\text{sw}}}{\rho_0 c_p} - \beta S_{\text{surf}}(E - P - R)\right) \tag{6}$$

Radiative buoyancy forcing:

$$\text{bosol} = g\alpha \frac{Q_{\text{sw}}}{\rho_0 c_p} \tag{7}$$

where:

- α = thermal expansion coefficient [1/K]
- β = haline contraction coefficient [1/psu]
- Q_{net} = net surface heat flux [W/m^2]
- Q_{sw} = shortwave radiation [W/m^2]
- $E - P - R$ = freshwater flux [m/s]

Implementation Notes:

- Positive heat flux = into ocean (warming)
- Positive freshwater flux = into ocean (freshening)
- `bo` drives turbulent mixing (non-penetrating)
- `bosol` penetrates with shortwave radiation

3.5 Interior Richardson Mixing

Purpose: Compute mixing coefficients in the ocean interior (below boundary layer) based on local Richardson number. This parameterizes shear instability and internal wave breaking.

Python Implementation:

- File: `KPP_ML/KPP_PY/kpp_routines.py`
- Function: `ri_iwmix()` (lines 100-250+)
- Called from: `kpp_core_driver.py::compute_mixing()` (line 205+)
- Richardson number: $\text{Ri} = N^2/S^2$ (line 120+)
- Diffusivity: $\kappa = \nu_0/(1 + a\text{Ri})^n$ (lines 140-180+)

Fortran Source:

- File: `pkg/kpp/kpp_routines.F`

- Function: `RI_IWMIX` (internal to KPPMIX, lines 1450-1600+)
- Called from: `KPPMIX` (line 500+)
- Richardson number: lines 1480-1510+
- Diffusivity formula: lines 1520-1560+

Key Computation:
Local Richardson number:

$$\text{Ri} = \frac{N^2}{S^2 + \epsilon} \tag{8}$$

Interior diffusivity (Pacanowski & Philander 1981 functional form):

$$\kappa = \frac{\nu_0}{(1 + a\text{Ri})^n} + \kappa_{\text{background}} \tag{9}$$

Typical values:

- $\nu_0 = 5.0\text{e-}3$ m^2/s (maximum diffusivity)
- $a = 5.0$ (empirical coefficient)
- $n = 2.0$ (empirical exponent)
- $\kappa_{\text{background}} = 1.0\text{e-}5$ m^2/s (background diffusivity)

Implementation Notes:

- Applied only below boundary layer (z < -hbl)
- Small ϵ regularizes division by zero when $S^2 = 0$
- Large Ri (stable) $\rightarrow$ small diffusivity
- Small Ri (shear instability) $\rightarrow$ large diffusivity

3.6 Boundary Layer Mixing (Shape Functions)

Purpose: Compute vertical profiles of mixing coefficients within the boundary layer using non-dimensional shape functions. This represents the structure of surface-driven turbulence.

Python Implementation:

- File: `KPP_ML/KPP_PY/kpp_scheme_specific.py`
- Function: `compute_bl_mixing()` (lines 200-400+)
- Called from: `kpp_core_driver.py::compute_mixing()` (line 220+)
- Velocity scales: `wscale()` computes w_m, w_s (lines 250-280+)
- Shape functions: cubic polynomial $G(\sigma)$ (lines 300-350+)
- Diffusivity: $\kappa(\sigma) = h \times w_s \times G(\sigma)$ (lines 360-390+)

Fortran Source:

- File: `pkg/kpp/kpp_routines.F`
- Function: `BLMIX` (internal to KPPMIX, lines 1050-1350+)
- Called from: `KPPMIX` (line 550+)
- Function: `WSCALE` (lines 900-1000+) computes velocity scales
- Shape function: lines 1150-1250+ (cubic profile)
- Diffusivity assembly: lines 1280-1330+

Key Computations:
Non-dimensional depth within boundary layer:

$$\sigma = -z/h_{\text{bl}} \quad (0 \leq \sigma \leq 1) \tag{10}$$

Cubic shape function (Large et al. 1994):

$$G(\sigma) = \sigma(1-\sigma)^2 \tag{11}$$

Velocity scales (Monin-Obukhov similarity, from lookup tables):

- $w_m(\sigma, \zeta)$ = turbulent velocity scale for momentum
- $w_s(\sigma, \zeta)$ = turbulent velocity scale for scalars
- $\zeta = -h_{\text{bl}}/L$ = stability parameter (L = Obukhov length)

Boundary layer diffusivity:

$$\kappa_{\text{BL}}(z) = h_{\text{bl}} \times w_s(\sigma, \zeta) \times G(\sigma) \tag{12}$$

Implementation Notes:

- Shape function peaks at $\sigma \approx 0.33$ (one-third depth)
- Goes to zero at surface ($\sigma = 0$) and base ($\sigma = 1$)
- Velocity scales vary with stability (stable vs. unstable forcing)
- Lookup tables (wmt, wst) precomputed for efficiency

3.7 Nonlocal Transport

Purpose: Compute the counter-gradient flux term (ghat) for scalars in the unstably-forced boundary layer. This represents large-eddy transport not captured by downgradient diffusion.

Python Implementation:

- File: `KPP_ML/KPP_PY/kpp_scheme_specific.py`
- Computed within: `compute_bl_mixing()` (lines 380-400+)
- Called from: `kpp_core_driver.py::compute_mixing()` (line 220+)
- Formula: $\gamma(\sigma) = c_s \times \sigma \times (1-\sigma)^2$ (line 390+)

- Applied when: unstable forcing (bo > 0)

Fortran Source:

- File: `pkg/kpp/kpp_routines.F`
- Function: `BLMIX` (lines 1050-1350+)
- Nonlocal term: lines 1200-1230+ (ghat computation)
- Shape function: $\gamma(\sigma)$ same cubic form

Key Computation:
Nonlocal transport shape function:

$$\gamma(\sigma) = c_s \times \sigma(1-\sigma)^2 \tag{13}$$

Nonlocal flux:

$$\text{ghat}(z) = \gamma(\sigma) \times \frac{\text{bo}}{w_s \times h_{\text{bl}}} \tag{14}$$

where:

- c_s = shape function coefficient (typically 10)
- `bo` = surface buoyancy forcing [m^2/s^3]
- w_s = turbulent velocity scale for scalars [m/s]
- Applied only for unstable forcing (bo > 0)

Implementation Notes:

- Nonlocal transport carries heat/salt vertically in convective boundary layers
- Represents counter-gradient flux (e.g., warm water transported downward in cooling conditions)
- Set to zero for stable forcing (bo < 0)
- Peak at $\sigma \approx 0.33$ (same as shape function)

3.8 Diffusivity Assembly

Purpose: Combine interior and boundary layer mixing coefficients into final vertical profiles, ensuring smooth transitions and applying physical constraints.

Python Implementation:

- File: `KPP_ML/KPP_PY/kpp_core_driver.py`
- Assembly logic: `compute_mixing()` (lines 230-270+)
- Bottom-to-top indexing: lines 240-250+ (interior below hbl, BL above)
- Re-indexing to top-of-cell: lines 260-270+ (MITgcm output convention)
- Background floor: applied at lines 265-268+

Fortran Source:

- File: `pkg/kpp/kpp_routines.F`
- Subroutine: `KPPMIX` (lines 28-1765)
- Assembly: lines 700-900+ (combine interior + BL)
- File: `pkg/kpp/kpp_calc.F`
- Export to model arrays: lines 550-650+ (`KPPviscAz`, `KPPdiffKzS`, `KPPdiffKzT`)

Key Operations:
For each vertical level:

$$\kappa_{\text{total}}(z) = \begin{cases} \kappa_{\text{BL}}(z) & \text{if } z > -h_{\text{bl}} \\ \kappa_{\text{interior}}(z) & \text{if } z \leq -h_{\text{bl}} \end{cases} \tag{15}$$

Background floor:

$$\kappa_{\text{final}}(z) = \max(\kappa_{\text{total}}(z), \kappa_{\text{background}}) \tag{16}$$

Implementation Notes:

- Boundary layer coefficients override interior coefficients above -hbl
- Background diffusivity (MITgcm's `diffKrNrS`, `diffKrNrT`) applied as floor
- Enhancement at interface (next step) adds discontinuity at -hbl
- Python uses bottom-to-top assembly then re-indexes for MITgcm compatibility

3.9 Enhancement at Interface

Purpose: Apply an additional enhancement to mixing coefficients at the boundary layer base to represent entrainment processes and ensure smooth transition to interior mixing.

Python Implementation:

- File: `KPP_ML/KPP_PY/kpp_scheme_specific.py`
- Function: `enhance_at_interface()` (lines 400-500+)
- Called from: `kpp_core_driver.py::compute_mixing()` (line 240+)
- Enhancement factor: $f_{\text{enh}} = 1+$ enhancement (lines 420-450+)
- Applied at: interface closest to -hbl (lines 460-480+)

Fortran Source:

- File: `pkg/kpp/kpp_routines.F`
- Function: `ENHANCE` (internal to KPPMIX, lines 1350-1450+)
- Called from: `KPPMIX` (line 600+)
- Enhancement logic: lines 1380-1420+

Key Computation:
Enhancement factor at boundary layer base:

$$f_{\text{enh}} = 1 + c_{\text{enh}} \times \delta \tag{17}$$

where:

- c_{enh} = enhancement coefficient (parameter)
- δ = nondimensional function of shear and stratification at interface

Enhanced diffusivity:

$$\kappa_{\text{enhanced}} = f_{\text{enh}} \times \kappa_{\text{BL}}(z = -h_{\text{bl}}) \tag{18}$$

Implementation Notes:

- Represents entrainment at boundary layer base
- Creates discontinuity in mixing profile at -hbl
- Applied only at the interface level (kbl in Fortran)
- Can be disabled by setting enhancement coefficient to zero

3.10 Double Diffusion (Optional)

Purpose: Add contributions from double-diffusive convection (salt fingering and diffusive convection) when temperature and salinity gradients have opposing effects on density.

Python Implementation:

- **Not implemented in 1D port** (excluded as not critical for validation)
- Would be in: `KPP_ML/KPP_PY/kpp_core.py` (if ported)

Fortran Source:

- File: `pkg/kpp/kpp_routines.F`
- Subroutine: `KPP_DOUBLEDIFF` (lines 1962-2121)
- Called from: `kpp_calc.F::KPP_CALC` (line 400+)
- Computes: additional diffusivity for salt fingering / diffusive convection

Implementation Notes:

- Salt fingering: warm, salty water over cool, fresh water
- Diffusive convection: cool, fresh water over warm, salty water
- Adds to diffusivity computed by main KPP scheme
- Not implemented in Python 1D port (3D feature)

4 Supporting Infrastructure

4.1 Parameter Handling

Python Approach:

- YAML-based configuration file in `configuration_yamls/`
- `kpp_default.yaml`: MITgcm default parameters
- Parameters loaded into `KPPParameters` dataclass
- All parameters have documented units and default values

Fortran Approach:

- Namelist-based configuration (`data.kpp` file)
- Namelist group: `KPP_PARM01`
- Parameters read in `kpp_readparms.F`
- Defaults defined in `KPP_PARAMS.h` header file
- Parameters stored in COMMON blocks

4.2 Grid and Coordinate System

Coordinate Conventions (both implementations):

- **z-axis**: Positive upward, z = 0 at surface
- **depth**: Positive downward (depth = $-z$)
- **Vertical index k**: k=0 is shallowest (surface), k=nz-1 is deepest (bottom)
- **Vertical spacing**: `dz[k]` or `cell_thickness[k]` is thickness of cell k

Staggering Conventions:

- Tracers (T, S) at cell centers
- Velocities (u, v) at cell centers (1D column)
- Mixing coefficients (κ_m, κ_h) at cell interfaces (top face)
- Nonlocal term (ghat) at cell interfaces (bottom face)
- Buoyancy gradients (dbloc) at cell interfaces

Python vertical arrays:

- `depth[k]`: depth of cell center k [m], positive
- `cell_thickness[k]`: thickness of cell k [m]
- `visc_az[k]`, `diff_kz_s[k]`, `diff_kz_t[k]`: mixing coefficients at top face of cell k [m^2/s]

- `ghat[k]`: nonlocal transport at bottom face of cell k [s/m²]
- Index 0 = surface face (set to 0)

Fortran vertical arrays:

- `rC(k)`: z-coordinate of cell center k [m]
- `rF(k)`: z-coordinate of cell interface k [m]
- `drF(k)`: thickness of cell k [m]
- `KPPviscAz(i,j,k)`, `KPPdiffKzS(i,j,k)`, `KPPdiffKzT(i,j,k)`: mixing coefficients at top face [m²/s]
- `KPPghat(i,j,k)`: nonlocal transport at bottom face [1/s]

4.3 Physical Constants

Common constants (both implementations):

- `gravity` (Python) = `gravity` (Fortran) = 9.81 m/s²
- `rho_const` (Python) = `rhoConst` (Fortran) = 1029.0 kg/m³
- `cp` = specific heat capacity = 3994.0 J/(kg K)

KPP-specific constants:

- `vonkarman` = von Kármán constant = 0.4
- `zref` = reference depth for bulk Ri = 10.0 m (typically surface layer)

4.4 Boundary Condition Details

Surface Boundary Conditions:

- Mixing coefficients: zero at surface (no surface diffusive flux)
- Surface forcing drives boundary layer depth and velocity scales
- Implemented by setting index 0 to zero

Bottom Boundary Conditions:

- Interior mixing continues to bottom
- No special boundary layer treatment at bottom (KPP is surface-forced)
- Bottom stress could be included (not implemented in 1D port)

No lateral boundaries in 1D column model

Python Module	MITgcm Fortran File(s)
`kpp_core_driver.py`	`pkg/kpp/kpp_calc.F` (main orchestrator)
`kpp_scheme_specific.py`	`pkg/kpp/kpp_routines.F` (BLDEPTH, BLMIX, ENHANCE)
`kpp_routines.py`	`pkg/kpp/kpp_routines.F` (RI_IWMIX, WSCALE, STATEKPP)
`kpp_shortwave.py`	`pkg/kpp/kpp_calc.F` (shortwave penetration)
`kpp_parameters.py`	`pkg/kpp/kpp_readparms.F`, `pkg/kpp/KPP_PARAMS.h`
`main/eos.py`	`pkg/kpp/kpp_routines.F::STATEKPP`, `model/src/find_rho.F`
Fortran files NOT ported (3D/horizontal features):	
—	`pkg/kpp/kpp_routines.F::KPP_DOUBLEDIFF` (double diffusion)
—	`pkg/kpp/kpp_routines.F::SMOOTH_HORIZ` (horizontal smoothing)
—	`pkg/kpp/kpp_routines.F::Z121` (vertical smoothing, not needed in 1D)

Table 1: Python-to-Fortran file correspondence for KPP port

5 File-to-File Correspondence Table

6 Quick Reference: Key Variables

6.1 Variable Name Mappings

6.2 Unit Conversions

Minor unit difference:

- **ghat (nonlocal transport)**:
  - Python: [s/m²] (following dimensional analysis from Large et al. 1994)
  - Fortran: [1/s] (MITgcm convention)
  - Conversion: Multiply by layer thickness when applying flux

All other variables use consistent SI units:

- Length: meters [m]
- Time: seconds [s]
- Temperature: degrees Celsius [°C]
- Salinity: practical salinity units [psu]
- Velocity: meters per second [m/s]
- Mixing coefficients: square meters per second [m²/s]

6.3 Sign Convention Differences

No sign convention differences between Python and Fortran implementations. Both use:

- z-axis positive upward (z = 0 at surface, z ¡ 0 below surface)
- depth positive downward (depth = $-z$)
- hbl positive (represents positive depth below surface)
- Positive buoyancy forcing = destabilizing (heating or freshening)
- Negative buoyancy forcing = stabilizing (cooling or salting)

Python Name	Fortran Name	Description
visc_az	KPPviscAz	Vertical viscosity [m^2/s]
diff_kz_s	KPPdiffKzS	Vertical diffusivity for salt [m^2/s]
diff_kz_t	KPPdiffKzT	Vertical diffusivity for temperature [m^2/s]
ghat	KPPghat	Nonlocal transport [s/m^2] (Python) vs. [1/s] (Fortran)
hbl	hbl	Boundary layer depth [m]
dbloc	dbloc	Local buoyancy gradient [m/s^2]
dvsq	dVsq	Velocity shear^2 relative to surface [$(m/s)^2$]
Ritop	Ritop	Numerator of bulk Ri [$(m/s)^2$]
ustar	ustar	Friction velocity [m/s]
bo	bo	Surface buoyancy forcing [m^2/s^3]
bosol	bosol	Radiative buoyancy forcing [m^2/s^3]
Rib	Rib	Bulk Richardson number [-]
kbl	kbl	Index of first level below hbl
depth	-rC	Depth (positive downward) [m]
cell_thickness	drF	Cell thickness [m]
Parameters:		
Ricr	KPPRicr	Critical Richardson number (default: 0.3)
cstar	KPPcstar	Shape function coefficient (default: 10)
epsilon	KPPepsilon	Nondim. extent of surface layer (default: 0.1)
Vtc	KPPVtc	Unresolved turbulence parameter (default: 1.8)
nu0	KPPnu0	Maximum interior diffusivity [m^2/s] (default: 5e-3)
diff_min	KPPdiffMin	Minimum diffusivity [m^2/s]
diff_max	KPPdiffMax	Maximum diffusivity [m^2/s]

Table 2: Python-to-Fortran variable name mappings